Binary Heap

Riwayat Versi

Versi	*Timestamp*	Halaman	Perubahan
1	09112026 - 2315	All	Rilis Pertama

Tujuan Pembelajaran

Setelah mempelajari modul, mahasiswa diharapkan mampu:

- Menjelaskan Binary Heap sebagai complete binary tree yang disimpan dalam array.
- Mengimplementasikan operasi insert, peek, dan penghapusan root dengan proses percolateUp dan percolateDown.
- Membedakan Max Heap dan Min Heap serta menentukan aturan prioritas yang menggunakan lebih dari satu kunci.
- Mengelola beberapa heap dengan aturan prioritas yang berbeda.
- Menelusuri perubahan array heap setelah pesanan masuk, root dikeluarkan, dan transaksi terjadi.
- Menggunakan heap untuk memilih elemen yang memenuhi aturan pemrosesan tertentu.
- Menganalisis kompleksitas waktu operasi heap dan keseluruhan proses pencocokan.

Catatan Awal

Untuk informasi mengenai pelaksanaan lab, metode penilaian, komponen soal, akses Judgels, serta tata cara melakukan submisi ke grader, silakan merujuk pada Panduan Lab SDA Gasal 2026/2027.

Daftar Isi

Riwayat Versi 1
Tujuan Pembelajaran 1
Catatan Awal 1
Daftar Isi 2
Persiapan Lab 3
- Prolog 3
- Bab 1: Binary Heap 3
- Bab 2: Operasi Utama dan Kompleksitas 4
  - Max Heap 5
  - Min Heap 8
- Bab 3: Konsep Lanjutan, Penelusuran, dan Edge Case 11
  - Satu Heap untuk Setiap Kelompok 11
  - Pembanding dan Tie-break 11
  - Operasi pada Dua Heap 11
- Epilog 12

Latihan Pra-Lab 13

Persiapan Lab

Prolog

Pada lab sebelumnya, Sofita berhasil merapikan pekerjaan Task Manager. Namun, jumlah tugas terus bertambah dan beberapa tugas harus dipilih lebih dahulu berdasarkan tingkat urgensinya. Jika Sofita memeriksa seluruh daftar setiap kali ingin memilih tugas terpenting, waktu pemrosesan akan membesar.

Detektif Jester menghadapi masalah serupa saat mengelola laporan kejadian. Ia perlu mengambil laporan dengan waktu paling awal, sementara Sofita perlu mengambil tugas dengan prioritas paling tinggi. Array biasa dapat menyimpan semua data, tetapi tidak langsung menyediakan elemen terdepan untuk kedua kebutuhan itu.

Struktur data yang dapat membantu mereka adalah **Binary Heap**. Dengan Max Heap, elemen bernilai terbesar dapat diakses dari root; dengan Min Heap, elemen terkecil dapat diakses dari root. Keduanya mendasari berbagai sistem antrean prioritas dan menjadi materi utama lab ini.

Bab 1: Binary Heap

Binary Heap adalah struktur data berbentuk pohon biner yang digunakan untuk mengelola elemen berdasarkan prioritas. Binary Heap harus menjaga dua sifat sekaligus

- **Sifat bentuk**: pohonnya berupa complete binary tree. Semua tingkat penuh, kecuali mungkin tingkat terakhir. Simpul pada tingkat terakhir diisi dari kiri ke kanan tanpa celah.
- **Sifat heap**: setiap parent memiliki prioritas setidaknya sama tinggi dengan anak-anaknya menurut aturan pembanding.

Sifat bentuk membuat tinggi pohon tetap **O(log n)** dan memungkinkan pohon disimpan rapat dalam array. Sifat heap membuat elemen yang paling diprioritaskan selalu berada di root. Jika salah satu sifat dilanggar, struktur tersebut bukan Binary Heap yang valid.

Heap biasanya disimpan dalam array, bukan dengan objek simpul dan pointer. Untuk indeks array yang dimulai dari 0:

Posisi	Rumus indeks
Parent dari indeks **i**	(i - 1) / 2
Left Child dari indeks i	2 * i + 1
Right Child dari indeks i	2 * i + 2

Rumus tersebut dapat ditulis sebagai helper kecil:

```java
int parentIndex(int i) { return (i - 1) / 2; }
int leftChild(int i)   { return 2 * i + 1; }
int rightChild(int i)  { return 2 * i + 2; }
```

Pembagian pada rumus parent adalah pembagian bilangan bulat. Indeks 0 adalah root dan tidak memiliki parent. Karena bentuk pohon complete, array tidak memiliki celah dan tidak memerlukan pointer kiri atau kanan

Heap tidak berarti seluruh elemen tersusun terurut di dalam array. Heap hanya menjamin bahwa root memiliki prioritas tertinggi dan setiap parent memiliki prioritas setidaknya setinggi anak-anaknya. Karena itu, membaca root dapat dilakukan dalam O(1), tetapi mencari elemen sembarang tetap dapat memerlukan O(n).

Contoh: array [60, 50, 40, 20, 35, 30] memenuhi sifat bentuk complete tree dan sifat Max Heap. Root-nya 60, sehingga itulah elemen yang langsung diketahui sebagai maksimum.

Contoh tersebut adalah Max Heap: 60 lebih besar dari kedua anaknya, 50 lebih besar dari 20 dan 35, serta 40 lebih besar dari 30. Namun 50 dan 40 tidak harus berada dalam urutan tertentu satu sama lain karena keduanya berada pada tingkat yang sama dan bukan parent-anak.

Operasi yang akan dibangun di bab berikutnya:

- **Insert**: tambahkan elemen baru tanpa merusak bentuk dan aturan heap.
- **Peek**: lihat elemen pada root tanpa mengubah heap.
- **Remove root**: keluarkan elemen terdepan dan pulihkan aturan heap.

Bab 2: Operasi Utama dan Kompleksitas

Bayangkan sebuah meja kerja tempat dokumen yang paling mendesak harus selalu mudah dijangkau. Max Heap menjaga nilai prioritas terbesar di bagian atas. Min Heap dapat dipakai pada sistem lain, misalnya untuk mengambil jadwal dengan waktu terkecil. Array di bawahnya tidak harus terurut penuh; heap hanya menjaga hubungan prioritas antara parent dan anak

Operasi utama pada Binary Heap:

- **Insert**: tambahkan elemen baru di ujung array, lalu panggil **percolateUp**.
- **Peek**: baca elemen pada root tanpa mengubah heap.
- **Remove root**: keluarkan elemen root, pindahkan elemen terakhir ke root, lalu panggil **percolateDown**.

Kompleksitas operasi: **peek** adalah **O(1)**, sedangkan **insert** dan **removeRoot** adalah **O(log n)**. Struktur array dipilih karena complete binary tree dapat disimpan tanpa pointer dan indeks parent-anak dapat dihitung langsung.

Berikutnya, operasi dijabarkan untuk Max Heap dan Min Heap. Setiap contoh kode adalah potongan method yang digabungkan ke kelas heap terkait.

Max Heap

Pada Max Heap, setiap parent memiliki nilai yang lebih besar daripada atau sama dengan anak-anaknya. Karena itu, root selalu berisi nilai maksimum.

```java
class MaxHeap {
    private final int[] heap;
    private int size;

    MaxHeap(int capacity) {
        heap = new int[capacity];
        size = 0;
    }

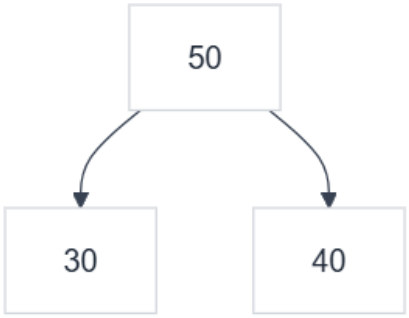
    boolean isEmpty() {
        return size == 0;
    }
```

1. Menambahkan Elemen (**insert**)
   Elemen baru ditempatkan di posisi kosong berikutnya di akhir array agar bentuk complete tree tetap terjaga. Lalu elemen dibandingkan dengan parent. Jika nilainya

lebih besar, tukarkan keduanya dan lanjutkan ke atas. Proses ini disebut **percolateUp**

Misalkan heap awal [50, 30, 40], kemudian kita memasukkan 60:

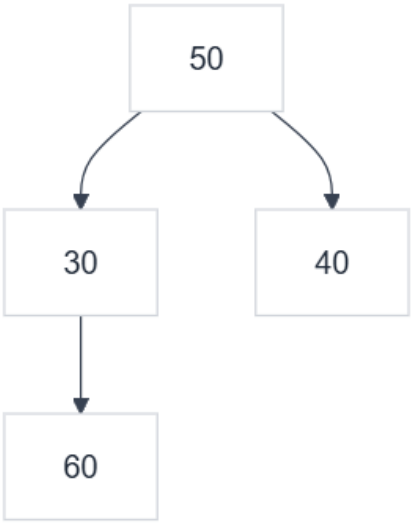
```
None
Sebelum: [50, 30, 40]
```



60 mula-mula ditambahkan di akhir array. Ia bertukar dengan 30, lalu dengan 50:

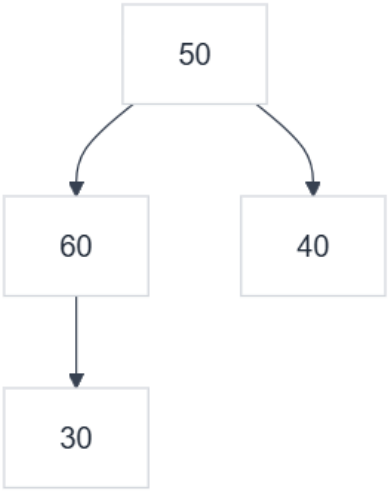
```
None
[50, 30, 40, 60]
```

Awalnya 60 menjadi anak kiri 30:



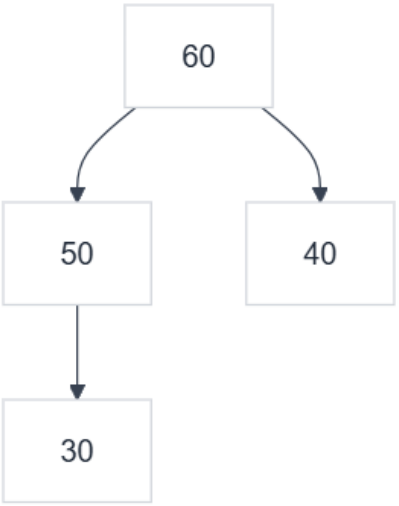
Setelah percolateUp satu tingkat, 60 bertukar dengan 30, sehingga array menjadi

```
None
[50, 60, 40, 30]
```



60 masih lebih besar daripada parent 50, sehingga bertukar sekali lagi. Hasil akhirnya:

```
None
[60, 50, 40, 30]
```



Elemen baru bergerak paling banyak setinggi pohon, sehingga **insert** membutuhkan **O(log n).**

Contoh Implementasi Kode

```java
 void insert(int value) {
        int i = size;
        heap[size++] = value;
        percolateUp(i);
    }

    private void percolateUp(int i) {
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (heap[parent] >= heap[i]) break;
            swap(parent, i);
            i = parent;
        }
    }
```

2. Melihat Root (**peek**)
   Nilai maksimum berada di indeks 0. Operasi peek hanya membaca heap[0], tidak mengubah heap, dan berjalan dalam O(1). Pastikan heap tidak kosong sebelum memanggilnya.

   Contoh Implementasi Kode

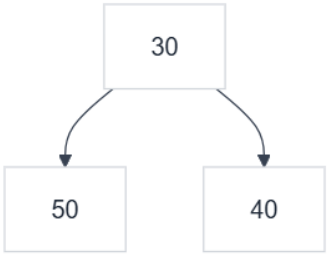
```java
int peek() {
        if (isEmpty()) throw new
IllegalStateException("Heap kosong");
        return heap[0];
    }
```

3. Mengeluarkan Root (**removeRoot**)
   Simpan nilai root sebagai hasil, pindahkan elemen terakhir ke indeks 0, lalu bandingkan dengan anak-anaknya. Tukarkan dengan anak yang lebih besar sampai aturan Max Heap kembali terpenuhi. Proses turun ini disebut **percolateDown**.

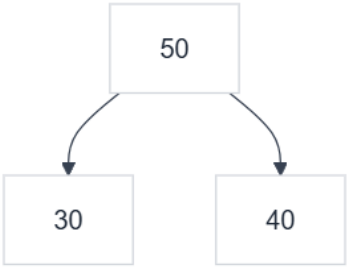
   Misalkan heap awal [60, 50, 40, 30], kemudian kita mengeluarkan Root:

```
None
Sebelum: [60, 50, 40, 30]
```

Sesudah 60 dikeluarkan dan 30 dipindahkan ke root, keadaan sementara masih melanggar aturan Max Heap:



Sehinnga anak dengan nilai terbesar adalah 50, jadi 30 bertukar dengannya. Pohon hasilnya:



Contoh Implementasi Kode

```java
int removeRoot() {
        if (isEmpty()) throw new
IllegalStateException("Heap kosong");
        int result = heap[0];
        size--;
        if (size > 0) {
            heap[0] = heap[size];
            percolateDown(0);
        }
        return result;
    }

    private void percolateDown(int i) {
        while (true) {
```

```
            int largest = i;
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            if (left < size && heap[left] > heap[largest])
largest = left;
            if (right < size && heap[right] >
heap[largest]) largest = right;
                  if (largest == i) break;
            swap(i, largest);
            i = largest;
        }
    }
```

4. Helper **Swap**

```
Java
private void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }
}
```

Min Heap

Pada Min Heap, setiap parent memiliki nilai yang lebih kecil daripada atau sama dengan anak-anaknya. Karena itu, root selalu berisi nilai minimum. Operasinya serupa dengan Max Heap, tetapi perbandingan nilainya dibalik.

```
Java
class MinHeap {
    private final int[] heap;
    private int size;

    MinHeap(int capacity) {
```

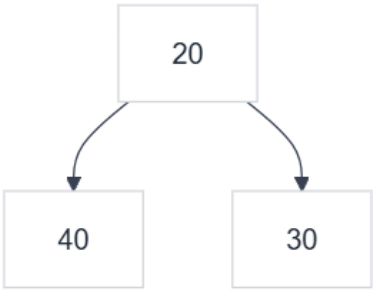
```
        heap = new int[capacity];
        size = 0;
    }

    boolean isEmpty() {
        return size == 0;
    }
```

1. Menambahkan Elemen (**insert**)
   Elemen baru ditempatkan di akhir array. Selama nilainya lebih kecil daripada parent, tukarkan keduanya. Proses ini disebut **percolateUp**.

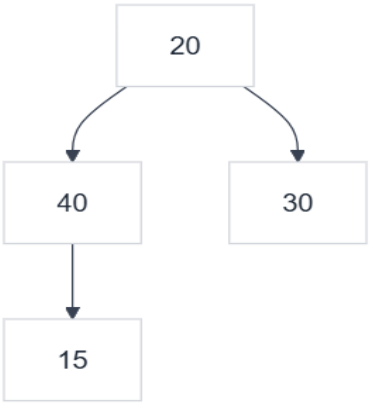
   Misalkan heap awal [20, 40, 30], kemudian kita memasukkan 15:

   ```
   None
   Sebelum: [20, 40, 30]
   ```

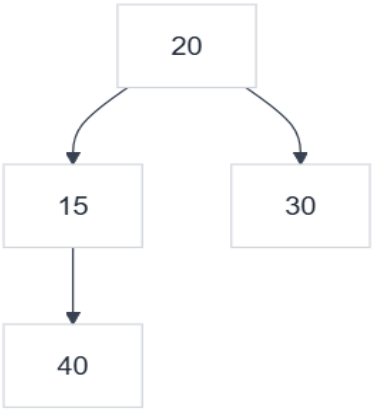

   ```
   None
   [20, 40, 30, 15]
   ```

   15 mula-mula menjadi anak kiri 40:



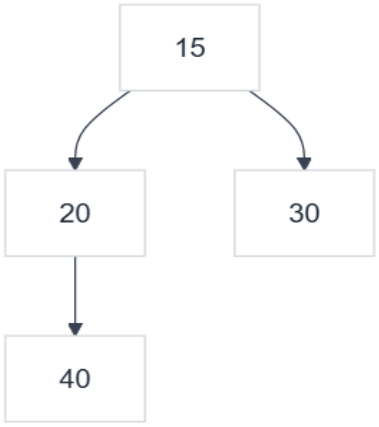
Karena 15 lebih kecil dari parent 40, keduanya bertukar. Array menjadi:

```
None
[20, 15, 30, 40]
```



15 masih lebih kecil daripada parent 20, maka keduanya bertukar lagi. Hasil akhirnya:

```
None
[15, 20, 30, 40]
```



Contoh Implementasi Kode

```java
    void insert(int value) {
        int i = size;
        heap[size++] = value;
        percolateUp(i);
    }

    private void percolateUp(int i) {
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (heap[parent] <= heap[i]) break;
            swap(parent, i);
            i = parent;
        }
    }
```

2. Melihat Root (**peek**)
   Nilai minimum berada di indeks 0. peek membaca heap[0] tanpa mengubah heap, sehingga berjalan dalam O(1). Pastikan heap tidak kosong sebelum memanggilnya.

   Contoh Implementasi Kode

```java
int peek() {
        if (isEmpty()) throw new
```

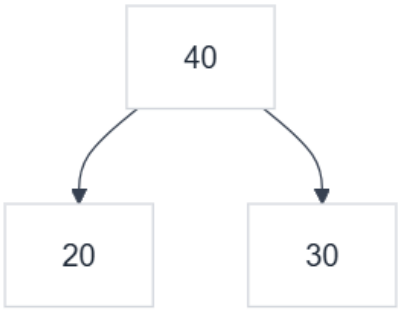
```
IllegalStateException("Heap kosong");
        return heap[0];
    }
```

3. Mengeluarkan Root (**removeRoot**)
   Simpan root sebagai hasil, pindahkan elemen terakhir ke root, lalu tukarkan dengan anak yang lebih kecil selama aturan Min Heap dilanggar.

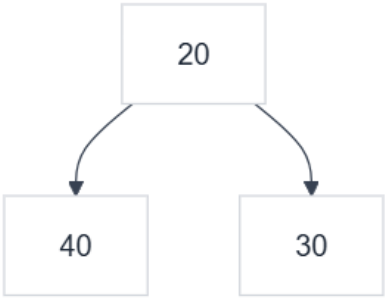
   Misalkan heap awal [15, 20, 30, 40], kemudian kita mengeluarkan Root:

```
None
Sebelum: [15, 20, 30, 40]
```

   Sesudah 15 dikeluarkan dan 40 dipindahkan ke root, keadaan sementara adalah:



   Anak yang lebih kecil adalah 20, jadi 40 bertukar dengannya. Pohon hasilnya:



   Kompleksitas removeRoot adalah **O(log n)**

   Contoh Implementasi Kode

```java
int removeRoot() {
        if (isEmpty()) throw new
IllegalStateException("Heap kosong");
        int result = heap[0];
        size--;
        if (size > 0) {
            heap[0] = heap[size];
            percolateDown(0);
        }
        return result;
    }
    private void percolateDown(int i) {
        while (true) {
            int smallest = i;
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            if (left < size && heap[left] <
heap[smallest]) smallest = left;
            if (right < size && heap[right] <
heap[smallest]) smallest = right;
            if (smallest == i) break;
            swap(i, smallest);
            i = smallest;
        }
    }
```

4. Helper **Swap**

```java
private void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }
}
```

Bab 3: Konsep Lanjutan, Penelusuran, dan Edge Case

Satu Heap untuk Setiap Kelompok

Satu program dapat mengelola beberapa kelompok elemen yang masing-masing memiliki aturan prioritas berbeda. Contohnya, sistem penjadwalan mungkin memisahkan tugas yang mendesak dan tugas rutin. Jika setiap kelompok perlu memilih elemen terdepan berdasarkan aturan sendiri, gunakan satu heap untuk setiap kelompok.

Satu kelompok dapat menggunakan Max Heap untuk memilih nilai kunci terbesar, sedangkan kelompok lain menggunakan Min Heap untuk memilih nilai terkecil. Kedua heap dapat memakai tipe elemen yang sama, tetapi pembandingnya berbeda. Root setiap heap tetap menyediakan elemen terdepan kelompok itu dalam O(1).

Pembanding dan Tie-break

Aturan pembanding harus menentukan urutan yang konsisten. Jika prioritas utama sama, gunakan kunci tambahan agar hasilnya tetap deterministik. Contohnya, ID atau waktu masuk dapat menjadi tie-break.

Untuk Max Heap, pembanding dapat mengutamakan kunci utama yang lebih besar dan, saat nilainya sama, ID yang lebih kecil. Untuk Min Heap, pembanding dapat mengutamakan kunci utama yang lebih kecil dan tetap memakai ID yang lebih kecil sebagai tie-break. Aturan lengkap ini harus digunakan oleh **insert** maupun **removeRoot**.

Jika pembanding berubah ketika elemen berada di dalam heap, susunan heap bisa tidak lagi benar. Dalam kasus itu, perbaiki posisi elemen atau keluarkan lalu masukkan kembali setelah nilainya berubah.

Operasi pada Dua Heap

Saat suatu proses mengelola dua heap, operasi utamanya adalah:

- **Peek**: baca root tanpa mengubah heap. Gunakan untuk menampilkan elemen terdepan atau memeriksa kondisi pasangan.
  ```
  if (!heap.isEmpty()) {
        int kandidat = heap.peek();
  }
  ```

- **Remove root:** keluarkan root, pindahkan elemen terakhir ke posisi root, lalu lakukan **percolateDown**
  ```
  if (!heap.isEmpty()) {
      int terdepan = heap.removeRoot();
  }
  ```

- **Insert**: masukkan elemen baru di akhir array, lalu lakukan **percolateUp**.
  `heap.insert(itemBaru);`

- **Cancel**: jika aturan masalah meminta elemen terdepan dikeluarkan, lakukan removeRoot. Jika heap kosong, keluaran atau state mengikuti spesifikasi masalah.

Epilog

Binary Heap menjaga elemen terdepan di root dengan mempertahankan bentuk complete tree dan aturan prioritas. Representasi array membuat hubungan parent-anak mudah dihitung, sementara **percolateUp** dan **percolateDown** menjaga heap setelah perubahan.

Max Heap dan Min Heap menyediakan dua arah prioritas. Dengan comparator yang tepat, tie-break yang konsisten, dan pemeriksaan kondisi kosong, struktur ini dapat dipakai untuk berbagai sistem pemrosesan prioritas.

Pada lab ini, Sofita akan menerapkan materi tersebut dengan dua heap, pembaruan kuantitas, dan penghapusan elemen terdepan. Pemahaman konsep dan penelusuran edge case pada bab-bab sebelumnya menjadi bekal sebelum mengimplementasikan soal lab.

Latihan Pra-Lab

Sofita kini ditugaskan mengurus sistem pelelangan tiker konser “Tabah” di Fasilkom. Sistem ini mempertemukan Fans yang ingin membeli tiket dengan Promotor yang menjual tiker.

Berbeda dengan sistem jual-beli instan, transaksi di bursa ini tidak terjadi secara otomatis saat pesanan masuk. Pesanan beli dan jual akan ditampung terlebih dahulu dan pencocokan (transaksi) hanya dilakukan ketika Sofita menekan tombol eksekusi

1. Antrean Beli (BID): Menyimpan pesanan dari Fans. Diprioritaskan berdasarkan **harga tertinggi**. Jika harga sama, pesanan yang datang lebih dahulu didahulukan.
2. Antrean Jual (OFFER): Menyimpan pesanan dari Promotor. Diprioritaskan berdasarkan **harga terendah**. Jika harga sama, pesanan yang datang lebih dahulu didahulukan.

Setiap pesanan (BID maupun OFFER) harus menggunakan satu penghitung ID global yang sama, yaitu bertambah 1 setiap kali pesanan baru dicatat (berurutan mulai dari 1).

Terdapat Q query yang diproses berurutan. Bantulah Sofita menentukan output dari setiap query.

Query pada Antrean Pesanan:

Query	Aksi	Output
`BID A B`	Fans mengajukan pesanan beli dengan harga maksimal A dan jumlah B tiket.	Mencetak ID Pesanan yang baru saja didapatkan dalam satu baris.
`OFFER A B`	Promotor mengajukan pesanan jual dengan harga minimal A dan jumlah B tiket.	Mencetak ID Pesanan yang baru saja didapatkan dalam satu baris.

Query pada Eksekusi dan Informasi

Query	Aksi	Output
`MATCH`	Sofita mengeksekusi pencocokan. Selama antrean BID dan OFFER tidak kosong, dan harga BID terdepan >= harga OFFER terdepan. transaksikan keduanya. Jumlah transaksi adalah nilai terkecil dari sisa tiker keduanya. Kurangi sisa tiket, harga transaksi menggunakan harga OFFER terdepan	Mencetak total tiket ditransaksikan dan total nilai transaksi, dipisahkan oleh satu spasi. Jika tidak ada transaksi, cetak 0 0.

`INFO`	Mengintip harga pesanan terdepan tanpa mengubah antrean	Mencetak harga BID terdepan dan harga OFFER terdepan, dipisahkan oleh satu spasi. Jika salah satu antrean kosong, harga untuk sisi tersebut cetak -1.

Ketentuan Pengerjaan

- Anda harus mengimplementasikan **Binary Max Heap** untuk pesanan beli dan **Binary Min Heap** untuk pesanan jual.
- Implementasi Binary Heap **harus dibuat sendiri** menggunakan **array atau ArrayList**. Boleh dibuat satu kelas heap yang dipakai dua kali dengan aturan perbandingan yang berbeda.
- Dilarang menggunakan struktur data siap pakai seperti **PriorityQueue**, **TreeMap**, **TreeSet**, atau struktur data koleksi lainnya sebagai pengganti heap.
- Dilarang menggunakan method sort bawaan seperti **Arrays.sort()**, **Collections.sort()**, **List.sort()**, atau sejenisnya.
- **Konsekuensi terhadap pelanggaran peraturan akan dikenakan sanksi poin akhir 0.**

Batasan Soal

- $1 \leq Q \leq 200.000$
- $1 \leq H \leq 10^9$
- $1 \leq J \leq 10^9$
- Total nilai transaksi pada satu query dapat mencapai 10^{18}, sehingga gunakan tipe data yang cukup besar (misalnya long pada Java).
- Time Limit: 2 detik

Format Masukan

```
Java

Q

query[0]

query[1]

...

query[Q-1]
```

Dengan:

- **Q** adalah jumlah query.
- query[i] adalah query ke-i, berbentuk BID A B, OFFER A B, MATCH, atau INFO sesuai format pada spesifikasi.

Format Keluaran

Untuk setiap query, cetak satu baris keluaran sesuai tabel pada deskripsi soal.

Contoh Masukan

```
Java
8
BID 500 10
OFFER 400 4
OFFER 400 3
BID 450 5
MATCH
INFO
MATCH
INFO
```

Contoh Keluaran

```
Java
1
2
3
4
7 2800
500 -1
0 0
500 -1
```

Penjelasan

- `BID 500 10`: Pesanan masuk ke antrean BID dan mendapat ID 1. Cetak 1.
- `OFFER 400 4 dan OFFER 400 3`: Dua pesanan masuk ke antrean OFFER dan berturut-turut mendapat ID 2 dan 3. Cetak 2, lalu cetak 3.
- `BID 450 5`: Pesanan masuk ke antrean BID dan mendapat ID 4. Cetak 4. Pada titik ini, BID terdepan adalah ID 1 (harga 500) dan OFFER terdepan adalah ID 2 (harga 400)
- `MATCH`: Transaksi ronde pertama terjadi antara BID ID 1 (sisa 10) dan OFFER ID 2 (sisa 4). Transaksi terbanyak 4 tiket pada harga 400 (nilai 1600), lalu ID 2 habis. Ronde kedua berlanjut antara sisa BID ID 1(sisa 6) dan OFFER ID 3 (sisa 3). Transaksi sebanyak 3 tiket pada harga 400 (nilai 1200), lalu ID 3 habis. MATCH berhenti karena antrean OFFER kosong. BID ID 1 masih di paling depan antrean dengan sisa 3 tiket. Total tiket = 7, total nilai = 2800. Cetak 7 2800.
- `INFO`: BID terdepan adalah pesanan ID 1 dengan harga 500. Antrean OFFER kosong, maka harus cetak -1. Cetak 500 -1
- `MATCH`: Tidak ada transaksi yang terjadi karena antrean OFFER kosong. Cetak 0 0.
- `INFO`: Kondisi antrean belum berubah. Cetak 500 -1

Solusi

Solusi Naif

Kode solusi naif:

```java
import java.util.*;


class Order {
    long id, price, amount;
    public Order(long id, long price, long amount) {
        this.id = id;
        this.price = price;
        this.amount = amount;
    }
}


public class Main {
    static List<Order> bidList = new ArrayList<>();
    static List<Order> offerList = new ArrayList<>();
    static long currentId = 1;


    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int q = sc.nextInt();


        while (q-- > 0) {
            String cmd = sc.next();
```

```
            if (cmd.equals("BID")) {
                long price = sc.nextLong();
                long amount = sc.nextLong();
                bidList.add(new Order(currentId, price, amount));
                System.out.println(currentId++);

            } else if (cmd.equals("OFFER")) {
                long price = sc.nextLong();
                long amount = sc.nextLong();
                offerList.add(new Order(currentId, price, amount));
                System.out.println(currentId++);

            } else if (cmd.equals("MATCH")) {
                long totalTickets = 0;
                long totalValue = 0;

                while (true) {
                    Order bestBid = getBest(bidList, true);
                    Order bestOffer = getBest(offerList, false);

                        if (bestBid != null && bestOffer != null &&
bestBid.price >= bestOffer.price) {
                            long trxAmount = Math.min(bestBid.amount,
bestOffer.amount);
                        totalTickets += trxAmount;
                        totalValue += trxAmount * bestOffer.price;
```

```
                        bestBid.amount -= trxAmount;
                        bestOffer.amount -= trxAmount;

                        // Hapus dari list jika tiketnya sudah habis
                        if (bestBid.amount == 0) bidList.remove(bestBid);
                        if (bestOffer.amount == 0) offerList.remove(bestOffer);
                    } else {
                        break;
                    }
                }
                System.out.println(totalTickets + " " + totalValue);

            } else if (cmd.equals("INFO")) {
                Order bestBid = getBest(bidList, true);
                Order bestOffer = getBest(offerList, false);

                long pBid = (bestBid == null) ? -1 : bestBid.price;
                long pOffer = (bestOffer == null) ? -1 : bestOffer.price;
                System.out.println(pBid + " " + pOffer);
            }
        }
        sc.close();
    }

    // Method pencarian linear O(N) untuk mencari prioritas tertinggi
```

```
    static Order getBest(List<Order> list, boolean isMax) {
        if (list.isEmpty()) return null;

        Order best = list.get(0);
        for (Order o : list) {
            if (isMax) {
                // Harga tertinggi, jika sama pilih ID terkecil
                  if (o.price > best.price || (o.price == best.price &&
o.id < best.id)) {
                    best = o;
                }
            } else {
                // Harga terendah, jika sama pilih ID terkecil
                  if (o.price < best.price || (o.price == best.price &&
o.id < best.id)) {
                    best = o;
                }
            }
        }
        return best;
    }
}
```

Ide dasar

Gunakan `ArrayList` untuk antrean BID maupun antrean OFFER.

- BID dan OFFER memakai .add() di akhir list untuk menambahkan pesanan baru beserta ID-nya.
- MATCH memakai fungsi pencarian linear (seperti `getBest()`) yang menelusuri seluruh elemen array untuk mencari pesanan dengan harga terbaik dan ID terkecil di kedua anteran.
- Penghapusan pesanan yang sudah habis transaksinya menggunakan `.remove()` pada object tersebut dari list.

Solusi ini menghasilkan keluaran yang benar secara logika, tetapi tidak memenuhi ketentuan soal karena menggunakan struktur dara list bawaan Java dan bukan mengimplementasikan Binary Heap buatan sendiri.

Analisis Inefisiensi

- Operasi pencarian mencari nilai maksimum atau minimum menggunakan linear scan membutuhkan O(N) untuk menelusuri N elemen.
- Operasi `.remove()` pada `ArrayList` menggeser seluruh elemen sesudahnya satu posisi ke kiri untuk menutup celah, yang juga membutuhkan waktu O(N).
- Pada kasus terburuk (misalnya Q = 200.000), loop MATCH yang berulang-ulang akan melakukan operasi pencarian dan pergeseran yang sangat masif, sehingga total operasinya mencapai kuadratik.
- Pendekatan naif ini sebenarnya sangat cepat saat menerima pesanan baru, karena `.add()` di ujung list berjalan dalam waktu O(1). Namun, ia mengorbankan operasi pencarian dan pengambilan menjadi O(N).

Perkiraan waktu

Kompleksitas solusi naif adalah $O(Q^2)$ pada kasus terburuk, karena setiap eksekusi di dalam MATCH dan INFO membutuhkan pencarian linear bernilai O(N), dan N dapat mencapai O(Q). Untuk batasan Q = 200.000, operasi kuadratiknya (200.000 x 200.000) akan menghasilkan sekitar 4×10^{10} (40 miliar) operasi. Angka ini jauh melampaui batas aman sekitar 10^8 hingga 2×10^8 operasi sederhana yang dapat dieksekusi Java dalam batas *Time Limit* 2 detik. Akibatnya, solusi ini dipastikan akan terkena TLE (*Time Limit Exceeded*).

Pendekatan Optimal

Antrean BID dibuat menggunakan Binary Max Heap, sedangkan antrean OFFER menggunakan Binary Min Heap, keduanya berbasis `ArrayList`. Penambahan pesanan baru diletakkan di akhir list lalu di-heapify up, sedangkan penghapusan pesanan teratas

memindahkan elemen terakhir ke root lalu di-heapify down. Karena elemen berprioritas tertinggi selalu terjamin berada di root (indeks 0), operasi INFO berjalan dalam O(1), sementara operasi penambahan dan penghapusan pesanan berjalan dalam O(log N).

Java

```
import java.io.*;
import java.util.*;


class Order {
    long id, price, amount;


    public Order(long id, long price, long amount) {
        this.id = id;
        this.price = price;
        this.amount = amount;
    }
}


class OrderHeap {
    private List<Order> heap;
     private boolean isMax; // true untuk BID (Max Heap), false untuk OFFER (Min Heap)


    public OrderHeap(boolean isMax) {
        this.heap = new ArrayList<>();
        this.isMax = isMax;
    }


    public boolean isEmpty() {
```

```
        return heap.isEmpty();
    }

    public Order peek() {
        if (heap.isEmpty()) return null;
        return heap.get(0);
    }

    public void add(Order order) {
        heap.add(order);
        percolateUp(heap.size() - 1);
    }

    public Order poll() {
        if (heap.isEmpty()) return null;
        Order best = heap.get(0);
        Order last = heap.remove(heap.size() - 1);
        if (!heap.isEmpty()) {
            heap.set(0, last);
            percolateDown(0);
        }
        return best;
    }

    // Mengembalikan true jika 'a' harus berada DI ATAS 'b' (prioritas lebih tinggi)
    private boolean hasHigherPriority(Order a, Order b) {
```

```
        if (isMax) { // Logika BID
            if (a.price != b.price) {
                return a.price > b.price;
            }
            return a.id < b.id; // Jika harga sama, ID terkecil menang
        } else { // Logika OFFER
            if (a.price != b.price) {
                return a.price < b.price;
            }
            return a.id < b.id; // Jika harga sama, ID terkecil menang
        }
    }

    private void percolateUp(int i) {
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (hasHigherPriority(heap.get(i), heap.get(parent))) {
                swap(i, parent);
                i = parent;
            } else {
                break;
            }
        }
    }

    private void percolateDown(int i) {
```

```
        int n = heap.size();
        while (true) {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            int best = i;

                    if (left < n && hasHigherPriority(heap.get(left),
heap.get(best))) {
                best = left;
            }
                    if (right < n && hasHigherPriority(heap.get(right),
heap.get(best))) {
                best = right;
            }

            if (best != i) {
                swap(i, best);
                i = best;
            } else {
                break;
            }
        }
    }

    private void swap(int i, int j) {
        Order temp = heap.get(i);
        heap.set(i, heap.get(j));
```

```
        heap.set(j, temp);
    }
}

public class Main {
    public static void main(String[] args) throws IOException {
                        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();

        String line = br.readLine();
        if (line == null) return;
        int q = Integer.parseInt(line.trim());

        OrderHeap bidHeap = new OrderHeap(true);  // Max Heap
        OrderHeap offerHeap = new OrderHeap(false); // Min Heap
        long currentId = 1;

        while (q-- > 0) {
            StringTokenizer st = new StringTokenizer(br.readLine());
            String cmd = st.nextToken();

            if (cmd.equals("BID")) {
                long price = Long.parseLong(st.nextToken());
                long amount = Long.parseLong(st.nextToken());
                bidHeap.add(new Order(currentId, price, amount));
                sb.append(currentId++).append("\n");
```

```
            }
            else if (cmd.equals("OFFER")) {
                long price = Long.parseLong(st.nextToken());
                long amount = Long.parseLong(st.nextToken());
                offerHeap.add(new Order(currentId, price, amount));
                sb.append(currentId++).append("\n");
            }
            else if (cmd.equals("MATCH")) {
                long totalTickets = 0;
                long totalValue = 0;

                while (!bidHeap.isEmpty() && !offerHeap.isEmpty()) {
                    Order bestBid = bidHeap.peek();
                    Order bestOffer = offerHeap.peek();

                    if (bestBid.price >= bestOffer.price) {
                            long trxAmount = Math.min(bestBid.amount,
bestOffer.amount);

                        totalTickets += trxAmount;
                        totalValue += trxAmount * bestOffer.price;

                        bestBid.amount -= trxAmount;
                        bestOffer.amount -= trxAmount;

                        if (bestBid.amount == 0) {
                            bidHeap.poll();
```

```
                        }
                        if (bestOffer.amount == 0) {
                            offerHeap.poll();
                        }
                    } else {
                        break;
                    }
                }
                                        sb.append(totalTickets).append("
").append(totalValue).append("\n");
            }
            else if (cmd.equals("INFO")) {
                Order bestBid = bidHeap.peek();
                Order bestOffer = offerHeap.peek();

                long pBid = (bestBid == null) ? -1 : bestBid.price;
                            long pOffer = (bestOffer == null) ? -1 :
bestOffer.price;

                                                sb.append(pBid).append("
").append(pOffer).append("\n");
            }
        }
        System.out.print(sb);
    }
}
```

Perbandingan

Approach	Kompleksitas Waktu	Kompleksitas Ruang	Catatan
Naif	BID, OFFER: O(1). MATCH, INFO: O(k · N). Total $O(Q^2)$ pada kasus terburuk.	O(Q)	Logika keluaran benar, tetapi dipastikan terkena TLE pada Q = 200.000 karena jauh melampaui *time limit*.
Optimal	BID, OFFER: O(log Q). MATCH: O(k log Q INFO: O(1). Total O(Q log Q).	O(Q)	Memenuhi semua ketentuan soal dan tereksekusi dengan aman di *time limit* 2 detik.